

Multi-Source Candidate Data Transformer

Technical Design Document — Stage 1

1. Pipeline

Seven sequential stages. Failures are isolated — a bad source is logged and skipped; the run always continues.

- detect — identify source type by filename / extension
- extract — each adapter maps its own field names to canonical paths, tagging every value with {source, method}
- normalize — phones → E.164, dates → YYYY-MM, country → ISO-3166 alpha-2, skills → canonical names; unparseable → None
- merge — group by email / profile URL / name+company; scalar winner by source priority; lists unioned & deduped
- score — noisy-OR over agreeing sources per field, capped at 0.99
- project — runtime JSON config reshapes the canonical record to the requested output schema
- validate — projected output checked before return; missing required field → clear error

2. Sources Handled

Structured: Recruiter CSV (recruiter_csv.py) · ATS JSON with its own field names (ats_json.py)

Unstructured: Resume prose — .txt / .pdf / .docx (resume.py) · GitHub profile via cached fixture for deterministic, offline runs (github.py)

3. Normalization

- Phones: phonenumbers lib → E.164; fallback regex for common US patterns; unparseable → None
- Dates: python-dateutil → YYYY-MM; year-only kept as YYYY; ambiguous → None
- Country: explicit ISO-3166 map; unrecognized → None, never guessed
- Skills: curated synonym table — js → JavaScript, react.js → React, postgres → PostgreSQL; unknowns kept as-is
- Emails: lower-cased and stripped; invalid format dropped

4. Merge Policy & Confidence Model

Identity keys (priority order): primary email → profile URL → name + current company (soft signal, flagged).

Scalar conflict winner: ats_json (0.90) > recruiter_csv (0.85) > resume (0.70) > github (0.60). Multi-valued fields are unioned after normalization so format-divergent duplicates collapse to one entry.

Confidence: confidence = $1 - \prod(1 - \text{weight}_i)$, capped at 0.99. overall_confidence = mean across populated core fields. Agreement raises a field's score; single-source values carry only that source's weight.

5. Configurable Output (Projection Layer)

The config is pure data — the same engine serves every output shape, no code changes needed. Example field spec:

```
{ "path": "phone", "from": "phones[0]", "type": "string", "normalize": "E164", "on_missing": "null" }
```

- **from** — index access (emails[0]), nested path (location.city), wildcard projection (skills[].name)
- **on_missing** — null (emit null) | omit (drop field) | error (abort with clear message)
- **include_confidence / include_provenance** — toggle those blocks globally or per field
- Output is validated against the projected schema via validation.py before being returned

6. Edge Cases

- Garbage / empty source — logged and skipped; run still succeeds with remaining sources
- Format-divergent duplicates — (415) 555-0132 and +1 415 555 0132 both normalize to +14155550132 → one entry
- Unparseable phone — dropped to None, never invented or guessed
- Partial dates — '2021' kept as YYYY; 'Jan 2017' → 2017-01; truly ambiguous → None
- Skill synonyms / casing — js, JS, JavaScript all canonicalize to 'JavaScript'
- on_missing: error with absent required field → clear validation error, not a silent crash